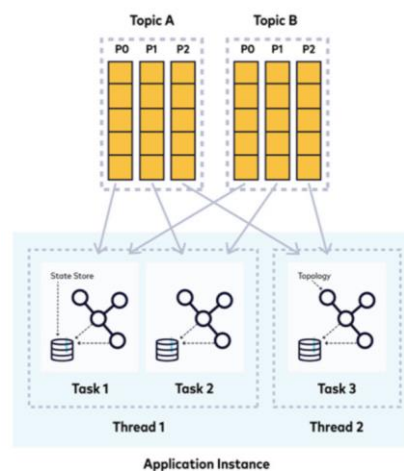


Data Streams with Kafka

Kafka Streams is a simple and light-weight client library for building real-time processing applications, where the input and output data is stored on Kafka clusters. It distinguishes between event time and processing time, offers support for windowing operations even in the case of out-of-order arrival of records, and transparently handles the load balancing of multiple instances of the same application by relying on the Kafka server-side technology.

Due to the intertwined nature of Kafka Streams and Kafka clusters, it is necessary to give an overview of the underlying architecture. Apache Kafka, the base system, offers access to cluster-stored data through producer and consumer libraries. Its main benefits are data parallelism, distributed coordination, and fault tolerance. The messaging layer of Kafka partitions data for storage and transport, while Kafka Streams partitions it for processing. In both cases, this mechanism allows for high performance, elasticity, scalability, and the aforementioned fault tolerance. Each Kafka Streams stream partition is a totally ordered sequence of records and maps to a Kafka topic partition. A data record from the stream maps to a message from the associated topic. The keys are the primary deciding factor regarding input split across partitions within a topic. The number of Kafka Streams tasks depends on the number of partitions of topics the application is subscribed to. Since each task is only responsible for its assigned list of partitions, the maximum number of tasks is bounded by the maximum number of partitions available. These tasks are handled by the stream threads in the application instances. The number of stream threads is customizable, but the default is one thread per instance. The assignment is taken care of automatically, with the recommendation of creating as many threads as there are partitions. In the case of excess, the unassigned threads remain idle. There is no overlap between information processed by independent threads. Application instances belonging to the same consumer group function as a complete unit where work is redistributed based on available members.



Another feature of Kafka Streams is the automatically managed state stores. They are created when performing actions such as joins, aggregation, or windowing. Every stream task in a Kafka Streams application may embed one or more local state stores that can be accessed via APIs to store and query data required for processing. Kafka Streams offers fault-tolerance and automatic recovery for such local state stores.

Fault tolerance is obtained through replication of the necessary system resources such as partitions, handled by the Kafka server-side technology, state store changelogs, managed by Kafka Streams. By harnessing both capabilities, failing tasks can be safely restarted and brought up to date, even when the tasks are moved to different application instances.

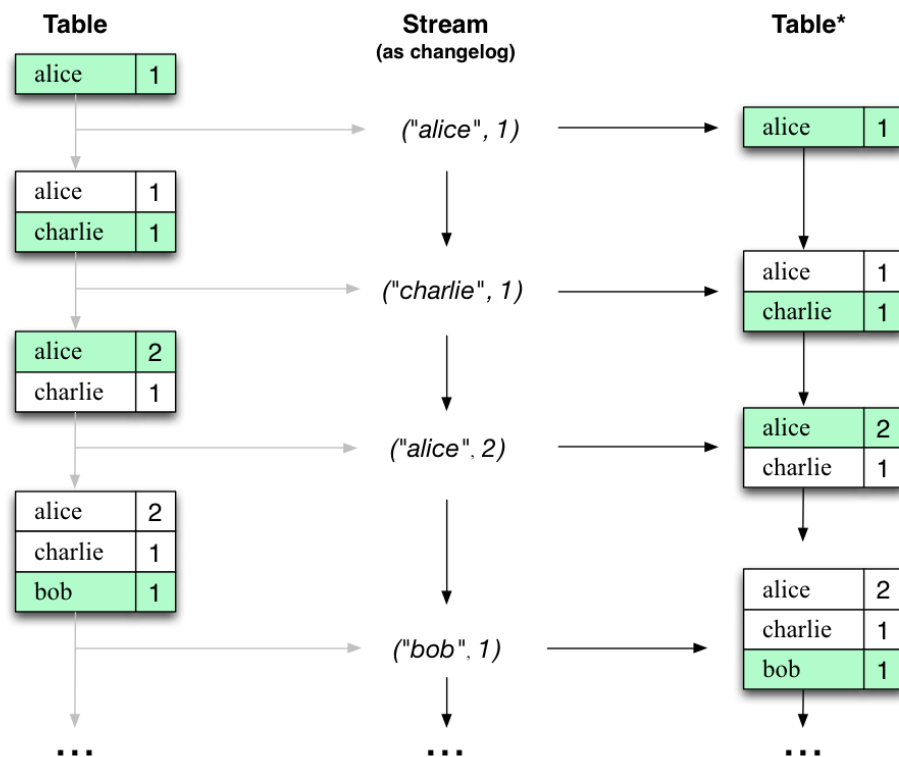
On a higher level, Kafka topics are the core abstraction for storing and processing data. Unlike traditional tables, Kafka relies on logs to record every event in a sequence. Simply put, all records are stored in order by appending new information to the internal logs. Every transformation will lead to the creation of a new topic instead of modifying existing records. In contrast to queues, Kafka messages remain available for multiple consumers to process independently. In the case of a failure, the system can easily reproduce the processing steps thanks to data persistence. Developers can opt to maintain only the latest information by enabling log compaction. Data retention time can also be modified as needed.

The fundamental building block of the processing mechanism is the stream. In Kafka Streams, it represents an unbounded and continuously updating data set. The stream records are immutable key value pairs with associated timestamps, allowing for the ordering of the stream. These data sets are passed through a processor topology composed of processing nodes, which receive one record at a time from the input stream and apply various operations, optionally producing records for downstream nodes, and edges, representing the streams themselves. Two types of processing nodes stand out due to their specific purpose: source and sink processors. Source processors do not have any upstream nodes; instead, they produce an input stream to their topology by consuming and forwarding records from one or more Kafka topics. The counterparts are the sink processors which send any received records from its upstream processors to a specified Kafka topic.

To properly define stream records, time must be taken into account. Kafka Streams distinguishes between event, processing and ingestion time. The event timestamp is associated with the moment when the event was created at the source, for example: in the case of a sensor, the assigned time is the time when the value was reported. In contrast, processing time is determined by the client system, particularly when the record is consumed. This can lead to discrepancies between the event time and the time reported by the stream processing application. A third notion is the ingested time, which represents the moment when a Kafka broker stores a certain record into a topic partition. The underlying Kafka configuration is responsible for deciding between event time and ingestion time, as the timestamps are automatically embedded. Kafka Streams leverages the `TimeStampExtractor` interface to retrieve these associated timestamps for

time-dependent operations such as windowing. The developer can modify the default behavior to, for example, retrieve time information directly from the message payload, as opposed to relying on the system-generated data. It is important to note that certain operations can also modify the timestamp assigned to a given record. A common scenario is applying aggregations and joins, which will generate a stream record with the maximum timestamp of the input stream records that contributed to the result. For stateless operations, the resulting record or records inhering the timestamp of the input.

When processing real-time data there are scenarios where stream records must be enriched with data stored in traditional databases. Kafka Streams allows for the seamless integration of both concepts. A stream can be considered a changelog of a table, where each data record in the stream captures a state change of the table. A stream can be easily turned into a “real” table by replaying the changelog from beginning to end to reconstruct the table. Similarly, in a more general analogy, aggregating data records in a stream will return a table. A table can be considered a snapshot, at a point in time, of the latest value for each key in a stream.



The stream-table duality is such an important concept that Kafka Streams models it explicitly via the `KStream`, `KTable`, and `GlobalKTable` interfaces.

An aggregation operation takes one input stream or table and yields a new table by combining multiple input records into a single output record. Examples of aggregations are computing counts or sums. In the Kafka Streams DSL, an input stream of an aggregation can be a `KStream`

or a KTable, but the output stream will always be a KTable. This allows Kafka Streams to update an aggregate value upon the out-of-order arrival of further records after the value was produced and emitted. When such out-of-order arrival happens, the aggregating KStream or KTable emits a new aggregate value. Because the output is a KTable, the new value is considered to overwrite the old value with the same key in subsequent processing steps.

Windowing lets you control how to group records that have the same key for stateful operations such as aggregations or joins into so-called windows. To wait for out-of-order records, a grace period can be specified for a given window representing the amount of time to wait after the window closes. If a record arrives after the grace period, it will be discarded. However, it is important to note that Kafka Streams considers the stream time, thus ignoring recording which could have been part of the window that arrived too late.

Regarding the interaction between out-of-order records and joins. Joins between streams handle them by default. For joins with tables, versioned state stores are required but can only handle out-of-order records on the stream side. However, in the case of table-table joins, the situation becomes a little more complicated. Without versioned state stores, out-of-order records are not handled explicitly, but the result of the join is a changelog stream which will eventually be consistent. With versioned stores, table-table join semantics change from offset-based to timestamp-based, thus leading to the proper management of these records.

As mentioned, processing code written in Kafka Streams is far more concise than the same code would be if written using the low-level Kafka clients. One way to examine their approaches for interacting with the log is to compare their corresponding APIs. The basic steps include: creating a producer and consumer and then subscribing to the single topic widgets. Next the records must be polled, and the ConsumerRecords collection is returned. The resulting records can be looped over, read, and filtered. To write the records to an output topic, a new ProducerRecord must be created for each one. In contrast, with Kafka Streams, only a StreamBuilder is necessary, along with the Java functional programming mechanisms, resulting in a more easily readable and maintainable code.

```

public static void main(String[] args) {
    try(Consumer<String, Widget> consumer = new KafkaConsumer<>(consumerProperties());
        Producer<String, Widget> producer = new KafkaProducer<>(producerProperties())) {
        consumer.subscribe(Collections.singletonList("widgets"));
        while (true) {
            ConsumerRecords<String, Widget> records =
consumer.poll(Duration.ofSeconds(5));
            for (ConsumerRecord<String, Widget> record : records) {
                Widget widget = record.value();
                if (widget.getColour().equals("red")) {
                    ProducerRecord<String, Widget> producerRecord = new
ProducerRecord<>("widgets-red", record.key(), widget);
                    producer.send(producerRecord, (metadata, exception)-> {.....} );
                }
            }
        }
    }
}

```

```

final StreamsBuilder builder = new StreamsBuilder();
builder.stream("widgets", Consumed.with(stringSerde, widgetsSerde))
    .filter((key, widget) -> widget.getColour.equals("red"))
    .to("widgets-red", Produced.with(stringSerde, widgetsSerde));

```

It is important to note that the Kafka Streams API is intended for usage with Java or Scala. In the practical project, a complementary python library was leveraged. Due to this aspect, the following section will refer to the adapted API in Quixstreams.

There are two primary objects: `StreamingDataFrame`, a predefined declarative pipeline to process and transform incoming messages, and `Application` to manage the Kafka-related setup, teardown and message lifecycle (consuming, committing). The `Application` consumes and deserializes messages, processes them with the `StreamingDataFrames`, produces the results to the output topic, automatically checkpoints processed messages and state for resiliency and scale using Kafka's built-in consumer groups mechanism.

To produce data to Kafka, an `Application` object is needed. It is the main entry point, and it provides API to create Producers, Topics, and other necessary objects. After the application is created, the developer can associate a topic, using this syntax:

`topic = app.topic(name='my_topic', value_serializer='json')`. If no such topic exists, the application can create it automatically. As indicated by the name, it corresponds to the Kafka topic. The value serializer is needed to maintain consistency across messages. The simplest

choice is the JSON serializer, which quickly converts a python dictionary into the value of a message. The topic serialize method receives the value and the key for the new message, in this way the developer has full control over the aspect of the event. To produce a message simply call the produce method and fill in the topic name, message key and message value. A producer with a default configuration can be retrieved with `application.get_producer()`, but the developer has the freedom to configure new producers as needed.

Processing data is done with `StreamingDataFrames` which follow a similar syntax to other data storage objects used by external Python libraries such as Pandas. Transformations are not limited to the provided API, instead they can be customized using developer-written methods. Among the basic features we enumerate updating data in-place, filtering, accessing and modifying data using column operations, windowed aggregations and as-is joins. The dataframe does not store the consumed data, it only constructs the processing plan which will be executed when the application is ran and messages are received. All operations performed on the same dataframe update its processing plan. Currently, branching is also supported using the assignment operation. If the dataframe has a different name and is within the scope of the application, it will be treated as a different plan. Quixstreams also provides mechanisms for joining split branches. This feature improves code flow by eliminating the need for multiple entry points or multithreaded processing, at the cost of reduced performance due to data duplication.

When processing incoming messages, columns can be added with newly calculated information or dropped as needed. For these scenarios, the `apply` method provides a versatile way to reformat resulting messages with the help of lambda functions. However, common operations have syntax shortcuts similar to Pandas. For complex transformations, in-place modifications on objects that are not dictionaries or calls for methods with side effects, such as printing, the API exposes the `update` method. The aforementioned methods can be used for filtering, but the straightforward way is to rely on the `filter` method which receives a function that should return `True` for messages to be kept. The messages passing through a dataframe can be written using the `to_topic()` method.

To aggregate results, two features are necessary: `group by` and windowing. The `group by` method functions like a key change for the resulting messages. From that point on, all the processors downstream will receive events with the new key. This method uses an internal topic for repartition and thus, the usage is limited to once per dataframe. Without relying on windows, aggregates can be computed manually for the resulting groups using `apply`. Quixstreams provides three types of windows: tumbling, hopping and, sliding. Tumbling windows slice time into non-overlapping intervals of a fixed size. Count-based Tumbling Windows slice incoming events into batches of a fixed size. Hopping windows slice time into overlapping intervals of a fixed size and with a fixed step. Count-based Hopping Windows slice incoming messages into overlapping batches of a fixed size with a fixed step. Sliding windows are overlapping time-based windows that advance with each incoming message, rather than at fixed time intervals like hopping windows. They have a fixed 1 ms resolution and perform better and are less resource-

intensive than hopping windows with a 1 ms step. Sliding windows do not produce redundant windows; every interval has a distinct aggregation. Sliding windows provide optimal performance for tasks requiring high-precision real-time monitoring. However, if the task is not time-critical or the data stream is extremely dense, tumbling or hopping windows may perform better. Sliding windows are overlapping windows that advance with each incoming message. They are equal to count-based hopping windows with a step of 1. Quix Streams supports 2 modes of emitting results for time windows: for each processed message in the given time window (current) or only once, after the time window is closed (final). To handle out-of-order events, the grace period mechanism is implemented in this library too. In addition, Quixstreams exposes a callback parameter to react to these late messages. In such a manner, they can be logged or rerouted to a different queue.

In Quixstreams, aggregation operations are divided into two groups: Aggregators and Collectors. Aggregators process each incoming message immediately and store only the running result (e.g. a running sum or count). The individual messages are not retained. Use them when the result can be computed incrementally without keeping all raw data, like counting items or summing values. Collectors store every raw message value in state until the window closes, at which point a final computation runs over all collected items. Use them when the final calculation requires all data at once, like calculating a median or producing a sorted list. Collectors are optimized for storing individual values to the state and perform significantly better than Aggregators when you need to accumulate values into a list. However, Collectors only support the final method. Aggregators allow you to select a column using the optional column parameter. When column is passed, the Aggregator will perform aggregation only over this column. It is assumed that the value is a dictionary. Otherwise, it will use the whole message. It is possible to calculate several different aggregations and collections over the same window. Custom classes can also be implemented as needed.

Use `StreamingDataFrame.join_asof()` to join two topics into a new stream where each left record is merged with the most recent right record that shares the same message key and has a timestamp less than or equal to the left record's timestamp. This join is built with the timeseries enrichment use cases in mind, where the left side represents some measurements and the right side represents events. During as-of join, the records on the right side get stored into a lookup table in the state, and the records from the left side query this state for matches. Join is a stateful operation, and it requires partitions of left and right topics to be assigned to the same application during processing. In all cases, messages with the same key must go to the same partition number for both left and right topics. This is Kafka's default behavior. Currently only inner and left joins are supported. Self-joins are not allowed.

Overall, developers can interact with Kafka clusters for stream processing through various methods. The default API is flexible due to the direct management of resources, but it is much more convoluted. To simplify the process, Kafka Streams API is available for Java/Scala and other libraries such as Quixstreams adapt the same principles in Python.

Sources:

<https://developer.confluent.io/courses/kafka-streams/get-started/>

<https://kafka.apache.org>

<https://www.baeldung.com/kafka-topics-partitions>

<https://www.geeksforgeeks.org/apache-kafka/introduction-to-apache-kafka-partitions/>

<https://developer.confluent.io/courses/apache-kafka/topics/>

<https://developer.confluent.io/courses/kafka-streams/internals/>

<https://quix.io/quix-streams>